

SQL JOINS: How to Choose the Right One

Complete Decision Guide with Examples

THE 4 MAIN JOIN TYPES

SQL JOIN TYPES	
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All from LEFT + matches from RIGHT
RIGHT JOIN	All from RIGHT + matches from LEFT
FULL JOIN	All rows from both tables (MySQL: use OR)

VISUAL EXAMPLES

INNER JOIN (Only Matching)

Left Table (A)

ID	Name
1	John
2	Jane
3	Bob
4	Alice

Right Table (B)

ID	Score
1	90
2	85
4	95
5	88

Result (A ⊕ B)

Only Matching

ID=1 ✓

ID=2 ✓

(ID=3, 4, 5 excluded)

LEFT JOIN (All from Left)

Left Table (A)

ID	Name
1	John
2	Jane
3	Bob
4	Alice

Right Table (B)

ID	Score
1	90
2	85
4	95
5	88

Result

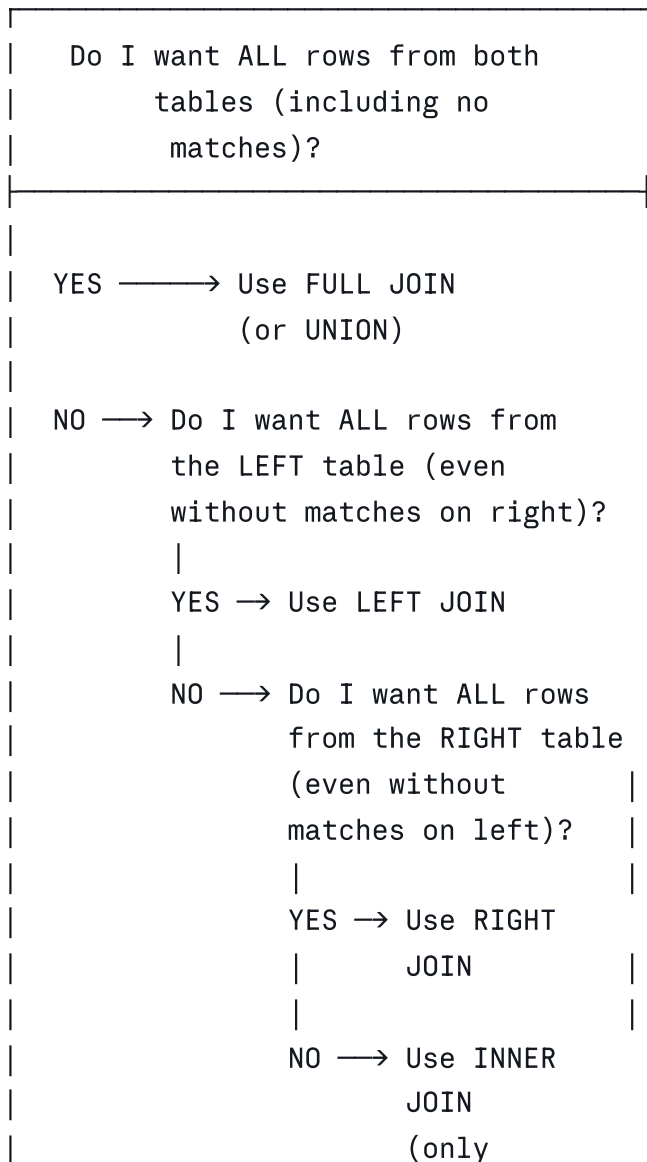
All LEFT + Match (ID=3,4 → NULL)		
1	John	90
2	Jane	85
3	Bob	NULL

		4	Alice	NULL
--	--	---	-------	------

RIGHT JOIN (All from Right)

Left Table (A)		Right Table (B)		Result		
ID	Name	ID	Score	Match + All RIGHT (ID=4,5 → NULL)		
1	John	1	90			
2	Jane	2	85	1	John	90
3	Bob	4	95	2	Jane	85
4	Alice	5	88	4	NULL	95
				5	NULL	88

DECISION FLOWCHART



```
| matching) |
```

PRACTICAL DECISION RULES

✅ Use INNER JOIN When:

- You only care about records that exist in BOTH tables
- You want to eliminate unmatched rows
- Examples:
 - "List shows that have genres" (show must have genre)
 - "List students with grades" (student must have a grade)
 - "Find users who made purchases" (user must have purchased)

SQL:

```
sql

SELECT s.title, g.name
FROM tv_shows s
INNER JOIN tv_show_genres g ON s.id = g.show_id;
-- Shows: Only shows that have genres
```

✅ Use LEFT JOIN When:

- You want ALL rows from the first (left) table
- Even if they don't have a match in the second table
- You expect some NULL values in the result
- Examples:
 - "List all shows, even those without genres" (show may have no genre)
 - "List all employees and their departments" (employee may have no dept)
 - "List all users with their last purchase date" (user may never purchased)

SQL:

```
SELECT s.title, g.genre_id
FROM tv_shows s
LEFT JOIN tv_show_genres g ON s.id = g.show_id
WHERE g.genre_id IS NULL;
-- Shows: Only shows WITHOUT genres (no match in right table)
```

✅ Use RIGHT JOIN When:

- You want ALL rows from the second (right) table
- Even if they don't have a match in the first table
- Examples:
 - "List all genres, even those with no shows" (rare, usually use LEFT instead)
 - "List all departments and their employees" (dept may have no employees)

SQL:

```
sql

SELECT g.name, s.title
FROM tv_shows s
RIGHT JOIN tv_show_genres g ON s.id = g.show_id;
-- Genres: All genres, even with no shows linked
```

✅ Use FULL JOIN When:

- You want ALL rows from both tables
- MySQL doesn't have FULL JOIN, so use UNION instead

SQL:

```
sql

SELECT s.id, s.title, g.genre_id
FROM tv_shows s
LEFT JOIN tv_show_genres g ON s.id = g.show_id
UNION
SELECT s.id, s.title, g.genre_id
FROM tv_shows s
RIGHT JOIN tv_show_genres g ON s.id = g.show_id;
-- All shows and all genres, even if no match
```

REAL-WORLD EXAMPLES

Example 1: Task 8 (Genre ID by Show)

Requirement: "List all shows that have at least one genre"

Decision:

- └ Do I want all shows? YES, but only with genres
- └ This means: Only shows that MATCH with genres
- └ Decision: INNER JOIN 

Query:

```
sql

SELECT s.title, g.genre_id
FROM tv_shows s
INNER JOIN tv_show_genres g ON s.id = g.show_id
ORDER BY s.title, g.genre_id;
```

Example 2: Task 9 (No Genre)

Requirement: "List shows WITHOUT a genre linked"

Decision:

- └ Do I want all shows? YES
- └ Even those without genres? YES (especially those!)
- └ Where should I find them? Not in tv_show_genres
- └ Decision: LEFT JOIN + WHERE genre_id IS NULL 

Query:

```
sql

SELECT s.title, g.genre_id
FROM tv_shows s
LEFT JOIN tv_show_genres g ON s.id = g.show_id
WHERE g.genre_id IS NULL
ORDER BY s.title;
```

Example 3: Task 10 (Number of Shows by Genre)

Requirement: "Count shows per genre (only genres with shows)"

Decision:

- └ Do I want all genres? NO, only those with shows
- └ This means: Only genres that MATCH with shows
- └ Decision: INNER JOIN 
- └ Plus: GROUP BY to count per genre

Query:

sql

```
SELECT g.name AS genre, COUNT(sg.show_id) AS number_of_shows
FROM tv_genres g
INNER JOIN tv_show_genres sg ON g.id = sg.genre_id
GROUP BY g.name
ORDER BY number_of_shows DESC;
```

Example 4: "All genres, even with no shows"

Requirement: "List all genres and count shows (even genres with 0 shows)"

Decision:

- └ Do I want ALL genres? YES
- └ Even without shows? YES
- └ Decision: LEFT JOIN 
- └ Plus: GROUP BY and COUNT

Query:

sql

```
SELECT g.name AS genre, COUNT(sg.show_id) AS number_of_shows
FROM tv_genres g
LEFT JOIN tv_show_genres sg ON g.id = sg.genre_id
GROUP BY g.name
ORDER BY number_of_shows DESC;
```

QUICK REFERENCE TABLE

JOIN Type	Returns	Use When	Result if No Match
INNER JOIN	Only matches	You want data from BOTH tables	Row excluded

JOIN Type	Returns	Use When	Result if No Match
LEFT JOIN	All from LEFT + matches	You want ALL LEFT rows, some RIGHT	NULL in RIGHT columns
RIGHT JOIN	Matches + all from RIGHT	You want ALL RIGHT rows, some LEFT	NULL in LEFT columns
FULL JOIN	All from both	You want EVERYTHING	NULL where no match

COMMON MISTAKES

✗ Mistake 1: Wrong JOIN for showing all records

```

sql
-- WRONG: Shows only those WITH genres
SELECT * FROM tv_shows
INNER JOIN tv_show_genres ON tv_shows.id = tv_show_genres.show_id;

-- CORRECT: Shows all, with NULL where no genre
SELECT * FROM tv_shows
LEFT JOIN tv_show_genres ON tv_shows.id = tv_show_genres.show_id;

```

✗ Mistake 2: Forgetting WHERE IS NULL

```

sql
-- WRONG: Still shows all shows
SELECT * FROM tv_shows
LEFT JOIN tv_show_genres ON tv_shows.id = tv_show_genres.show_id;

-- CORRECT: Only shows without genres
SELECT * FROM tv_shows
LEFT JOIN tv_show_genres ON tv_shows.id = tv_show_genres.show_id
WHERE tv_show_genres.genre_id IS NULL;

```

✗ Mistake 3: Using wrong table in WHERE

```

sql

```

```
-- WRONG: Genre condition filters before join
SELECT * FROM tv_shows s
INNER JOIN tv_show_genres g ON s.id = g.show_id
WHERE s.genre_id = 5; -- show doesn't have genre_id!

-- CORRECT: Genre condition on joined table
SELECT * FROM tv_shows s
INNER JOIN tv_show_genres g ON s.id = g.show_id
WHERE g.genre_id = 5; -- correct column from joined table
```

DECISION CHECKLIST

Before writing a JOIN, ask yourself:

1. ☐ Do I need rows that don't match?
 - └ Yes → Use LEFT or RIGHT or FULL
 - └ No → Use INNER
2. ☐ If yes, from which table do I want ALL rows?
 - └ Left table → Use LEFT JOIN
 - └ Right table → Use RIGHT JOIN
 - └ Both tables → Use FULL JOIN (or UNION)
3. ☐ Do I need to filter for NULL values?
 - └ Yes → Add WHERE column IS NULL
 - └ No → Continue
4. ☐ Do I need to group/aggregate?
 - └ Yes → Add GROUP BY
 - └ No → Done!
5. ☐ Is my result correct?
 - └ Yes → Commit! 
 - └ No → Check your ON condition

TASKS 0-11 JOIN USAGE SUMMARY

Task	Type	Reason
0-5	N/A	No JOINS needed
6	GROUP BY	Group temps by city

Task	Type	Reason
7	GROUP BY	Find max temp per state
8	INNER JOIN	Only shows WITH genres
9	LEFT JOIN	Shows WITHOUT genres
10	INNER JOIN	Only genres WITH shows
11	INNER JOIN	Only shows WITH ratings

Remember:

-  **INNER JOIN** = Intersection (only matches)
-  **LEFT JOIN** = All from left + matches
-  **RIGHT JOIN** = Matches + all from right
-  **FULL JOIN** = Everything (use UNION in MySQL)

Last Updated: May 8, 2026 **Status:**  Complete JOIN Decision Guide